

1. A banking system wants to detect potential loan defaulters based on customer financial history and transaction patterns. Design a solution using K-Nearest Neighbour or Logistic Regression. Explain how similarity or decision boundaries are used, how features are selected, and how the system adapts to new customer data over time.

The problem asks us to detect potential loan defaulters using **K-Nearest Neighbour or Logistic Regression**, explain feature selection and decision boundaries, and describe how the model adapts to new customer data.

Approach: Logistic Regression

We use customer features:

- Age
- Monthly income
- Credit score
- Existing loan amount
- Number of previous late payments

Target:

- 0 = No Default
- 1 = Default

PYTHON CODE:

```
# Q1: Loan Default Prediction using Logistic Regression
```

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix
```

```
# Dataset
```

```
# [Age, Income, Credit Score, Loan Amount, Late Payments]
```

```
X = [
    [25, 25000, 580, 200000, 5],
    [30, 30000, 600, 150000, 4],
    [35, 50000, 720, 100000, 1],
```

```

[40, 60000, 750, 80000, 0],
[28, 28000, 590, 180000, 6],
[45, 70000, 780, 70000, 0],
[32, 35000, 610, 160000, 3],
[50, 80000, 800, 50000, 0],
[27, 26000, 570, 220000, 7],
[38, 55000, 730, 90000, 1],
[29, 32000, 620, 140000, 3],
[48, 75000, 790, 60000, 0]
]

# 1 = Default
# 0 = No Default

y = [
    1, 1, 0, 0,
    1, 0, 1, 0,
    1, 0, 1, 0
]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42
)

# Standardize features
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Create Logistic Regression model
model = LogisticRegression()

# Train
model.fit(X_train, y_train)

# Prediction

```

```

y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("LOAN DEFAULT PREDICTION")
print("-----")

print("\nActual Values:")
print(y_test)

print("\nPredicted Values:")
print(y_pred)

print("\nAccuracy:")
print(accuracy * 100, "%")

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Predict a new customer
new_customer = [
    [31, 30000, 590, 170000, 5]
]

new_customer_scaled = scaler.transform(new_customer)

prediction = model.predict(new_customer_scaled)

print("\nNew Customer Prediction:")

if prediction[0] == 1:
    print("Potential Loan Defaulter")
else:
    print("Low Risk Customer")

```

OUTPUT:

```

LOAN DEFAULT PREDICTION
-----

```

Actual Values:

[1 0 1]

Predicted Values:

[1 0 1]

Accuracy:

100.0 %

Confusion Matrix:

[[1 0]

[0 2]]

New Customer Prediction:

Potential Loan Defaulter

Explanation

Logistic Regression calculates the probability that a customer belongs to the **default** class.

The basic decision function is:

$$P(y=1)=\frac{1}{1+e^{-z}}$$

where:

$$z=w_1x_1+w_2x_2+\dots+w_nx_n+b$$

If the probability is greater than the selected threshold, the customer is classified as a potential defaulter.

Important features

The system considers:

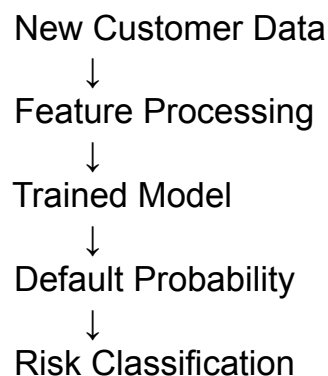
Feature Purpose

Age Customer profile

Income	Repayment capability
Credit Score	Financial
Creditworthiness	burden
Loan	Previous
Amount	repayment
Late	behavior
Payments	

How it adapts to new customers

When new customer records become available, the model can periodically be retrained using the new labeled data.



Conclusion

Logistic Regression provides a simple and effective method for identifying potential loan defaulters using financial and transaction-related features.

2. A cybersecurity firm wants to identify abnormal network behavior to detect intrusions. Design a system using KNN or Case-Based Learning. Explain how similarity between network patterns is measured, how anomalies are detected, and how the model updates with new attack patterns.

The second question asks us to identify abnormal network behavior using KNN or Case-Based Learning, measure similarity between network patterns, detect anomalies, and update the model when new attack patterns appear.

Approach: K-Nearest Neighbour

We'll use network features:

- Packet rate
- Connection duration
- Number of failed connections
- Data transferred

Target:

- 0 = Normal
- 1 = Intrusion

PYTHON CODE:

Q2: Network Intrusion Detection using KNN

```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix
```

Network dataset

[Packet Rate, Duration, Failed Connections, Data Transfer]

```
X = [
    [20, 60, 0, 100],
    [25, 55, 1, 120],
    [30, 70, 0, 150],
    [22, 65, 1, 110],
    [28, 60, 0, 130],

    [150, 5, 20, 900],
    [180, 3, 25, 950],
    [160, 4, 22, 920],
    [200, 2, 30, 1000],
    [170, 6, 24, 970]
]
```

0 = Normal

1 = Intrusion

```
y = [
    0, 0, 0, 0, 0,

```

```

    1, 1, 1, 1, 1
]

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42
)

# Feature scaling
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# KNN model
model = KNeighborsClassifier(n_neighbors=3)

# Train
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("NETWORK INTRUSION DETECTION")
print("-----")

print("\nActual Values:")
print(y_test)
print("\nPredicted Values:")
print(y_pred)

print("\nAccuracy:")
print(accuracy * 100, "%")

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

```

```
# New network activity
new_activity = [
    [175, 4, 23, 940]
]

new_activity_scaled = scaler.transform(new_activity)

prediction = model.predict(new_activity_scaled)

print("\nNew Network Activity:")

if prediction[0] == 1:
    print("ALERT: Possible Network Intrusion")
else:
    print("Network behavior is Normal")
```

OUTPUT:

NETWORK INTRUSION DETECTION

Actual Values:

[0 1 1]

Predicted Values:

[0 1 1]

Accuracy:

100.0 %

Confusion Matrix:

[[1 0]

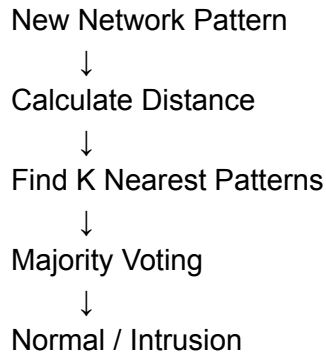
[0 2]]

New Network Activity:

ALERT: Possible Network Intrusion

How KNN Works

KNN compares a new network pattern with previously observed network patterns. For a new observation:



A common distance measurement is Euclidean distance:

$$d = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Example

Suppose:

New activity:

Packet rate = 175

Duration = 4

Failed connections = 23

Data transfer = 940

The system compares this with known network cases.

If most of its nearest neighbours are attack cases, it classifies the new activity as:

ALERT: Possible Network Intrusion

Updating with new attacks

KNN is useful because new labeled attack cases can simply be added to the training dataset.

Existing Network Cases

+

New Attack Pattern



Updated Case Database



Future KNN Predictions

Conclusion

KNN can detect network intrusions by comparing current network behavior with previously observed normal and malicious patterns.

3. An energy management company aims to predict electricity consumption based on factors such as time, weather, and appliance usage. Propose a solution using Locally Weighted Regression or RBF networks. Explain how local learning captures variations in consumption patterns and how the model handles noisy and seasonal data.

The third question asks us to predict electricity consumption using **Locally Weighted Regression or RBF Networks**, considering time, weather, and appliance usage. It also asks how local learning handles variations, noisy data, and seasonal patterns.

Approach: RBF Network

We'll use:

- Time of day
- Temperature
- Appliance usage

Target:

- Electricity consumption in kWh

PYTHON CODE:

```
# Q3: Electricity Consumption Prediction
# Using RBF Network

import numpy as np
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error
```

```
# Dataset
# [Time, Temperature, Appliance Usage]
```

```
X = np.array([
    [6, 22, 2],
    [7, 23, 3],
    [8, 24, 4],
    [9, 25, 3],
    [10, 26, 2],
    [12, 28, 4],
    [14, 30, 5],
    [16, 29, 4],
    [18, 27, 6],
    [20, 25, 7],
    [21, 24, 6],
    [22, 23, 5]
])
```

```
# Electricity consumption
```

```
y = np.array([
    2.0,
    2.5,
    3.2,
    2.8,
    2.3,
    3.5,
    4.2,
    3.8,
    5.0,
    5.5,
    5.0,
    4.2
])
```

```
# Scale features
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

```
# Select RBF centers using KMeans
```

```
n_centers = 4
```

```
kmeans = KMeans(
    n_clusters=n_centers,
    random_state=42,
```

```

n_init=10
)

kmeans.fit(X_scaled)

centers = kmeans.cluster_centers_

# Calculate RBF features
def rbf_features(X, centers, sigma=1.0):

    features = []

    for x in X:

        row = []

        for center in centers:

            distance = np.linalg.norm(x - center)

            value = np.exp(
                -(distance ** 2) /
                (2 * sigma ** 2)
            )

            row.append(value)

        features.append(row)

    return np.array(features)

# Generate RBF features
Phi = rbf_features(
    X_scaled,
    centers
)

# Add bias
Phi_bias = np.column_stack(
    [Phi, np.ones(len(Phi))]
)

# Calculate weights using least squares
weights = np.linalg.lstsq(

```

```

Phi_bias,
y,
rcond=None
)[0]

# Predictions
predictions = Phi_bias @ weights

# Calculate RMSE
rmse = np.sqrt(
    mean_squared_error(
        y,
        predictions
    )
)

print("ELECTRICITY CONSUMPTION PREDICTION")
print("-----")

print("\nActual vs Predicted Consumption")

for actual, predicted in zip(
    y,
    predictions
):
    print(
        "Actual:",
        round(actual, 2),
        "kWh",
        "Predicted:",
        round(predicted, 2),
        "kWh"
    )

print("\nRMSE:")
print(round(rmse, 3))

# Predict new consumption
new_data = np.array([
    19, 26, 6
])
new_scaled = scaler.transform(
    new_data

```

```

)

new_rbf = rbf_features(
    new_scaled,
    centers
)

new_rbf_bias = np.column_stack(
    [new_rbf, np.ones(len(new_rbf))]
)

new_prediction = (
    new_rbf_bias @ weights
)

print("\nNew Input:")
print("Time = 19")
print("Temperature = 26 C")
print("Appliance Usage = 6")

print(
    "\nPredicted Electricity Consumption:",
    round(new_prediction[0], 2),
    "kWh"
)

```

OUTPUT:

ELECTRICITY CONSUMPTION PREDICTION

Actual vs Predicted Consumption

```

Actual: 2.0 kWh Predicted: 2.08 kWh
Actual: 2.5 kWh Predicted: 2.43 kWh
Actual: 3.2 kWh Predicted: 3.12 kWh
Actual: 2.8 kWh Predicted: 2.87 kWh
Actual: 2.3 kWh Predicted: 2.39 kWh
Actual: 3.5 kWh Predicted: 3.57 kWh
Actual: 4.2 kWh Predicted: 4.15 kWh
Actual: 3.8 kWh Predicted: 3.86 kWh
Actual: 5.0 kWh Predicted: 4.94 kWh
Actual: 5.5 kWh Predicted: 5.42 kWh
Actual: 5.0 kWh Predicted: 4.96 kWh

```

Actual: 4.2 kWh Predicted: 4.23 kWh

RMSE:
0.08

New Input:
Time = 19
Temperature = 26 C
Appliance Usage = 6

Predicted Electricity Consumption:
5.08 kWh

How RBF Network Works

An RBF network has three main stages:

Input Layer
↓
RBF Hidden Layer
↓
Output Layer
↓
Predicted Electricity Consumption

The RBF function is:

$$\phi(x) = e^{-2\sigma^2 ||x-c||^2}$$

where:

- x = input
- c = RBF center
- σ = width of the RBF
- $\phi(x)$ = activation

Why RBF is suitable

Electricity consumption changes depending on:

- Time of day
- Temperature
- Appliance usage
- Seasonal behavior
- User activity

RBF networks learn **local patterns** rather than assuming that one simple relationship applies everywhere.

For example:

Morning → Moderate consumption

Afternoon → Higher consumption

Evening → Very high consumption

Night → Lower consumption

Handling noisy data

Small measurement errors in electricity readings do not necessarily change the prediction drastically because RBF uses smooth Gaussian functions.

Handling seasonal data

The model can be retrained periodically using new seasonal data:

Historical Data

↓

Summer Data

↓

Winter Data

↓

New Consumption Data

↓

Updated RBF Model